ES-CS301 — Data Structure & Algorithms

Semester III • 3L:0T:4P • 3 Credits

Module 2 Exam Notes: Stacks and Queues: Principles, Operations and Applications

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 Abstract Data Type Stack and Fundamental Operations **2**
1.1 Array Implementation of Stack 2
2 Applications of Stacks **3**
2.1 Arithmetic Expression Formats and Operator Precedence 3
2.2 Infix to Postfix Conversion Algorithm 4
2.3 Evaluation of Postfix Expressions 5
2.4 Infix to Prefix Conversion and Prefix Evaluation 6
2.5 Balanced Delimiter Checking and Call Stack Simulation 7
3 Abstract Data Type Queue and Linear Queue **7**
3.1 Array Implementation of Linear Queue 7
4 Advanced Queue Architectures **8**
4.1 Circular Queue 8
4.2 Priority Queue 9
4.3 Double-Ended Queue (DEQUE) 10

1. Abstract Data Type Stack and Fundamental Operations

Definition: Stack Abstract Data Type (ADT)

A **Stack** is a linear, restricted-access data structure adhering strictly to the **Last-In, First-Out (LIFO)** or **First-In, Last-Out (FILO)** operating discipline. All insertions (`push`) and deletions (`pop`) are constrained to take place at a single end termed the **Top of Stack (TOS)**.

Formally, a Stack S over an element domain E is defined by:

- State: An ordered sequence $S = \langle e_0, e_1, \dots, e_k \rangle$ where e_k is the Top element.
- Auxiliary Pointer: An integer index top $\in [-1, \text{MAX} - 1]$ indicating the index of the most recently inserted item.

1.1 Array Implementation of Stack

In an array-based static implementation, elements reside in contiguous storage $S[0 \dots \text{MAX}-1]$.

- **Initial Condition:** top $= -1$ (stack is strictly empty).
- **Stack Overflow Condition:** Attempting to insert an element when the physical capacity is exhausted:

$$\text{Overflow} \iff \text{top} == \text{MAX} - 1 \tag{1}$$

- **Stack Underflow Condition:** Attempting to retrieve or delete an element from an empty stack:

$$\text{Underflow} \iff \text{top} == -1 \tag{2}$$

Core Stack Primitive Algorithms and Complexity Bounds

1. Algorithm Push(S, top, x):

```
if (top == MAX - 1)
    report "Stack Overflow" and exit error;
top = top + 1;
S[top] = x;
```

Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

2. Algorithm Pop(S, top):

```
if (top == -1)
    report "Stack Underflow" and exit error;
item = S[top];
top = top - 1;
return item;
```

Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

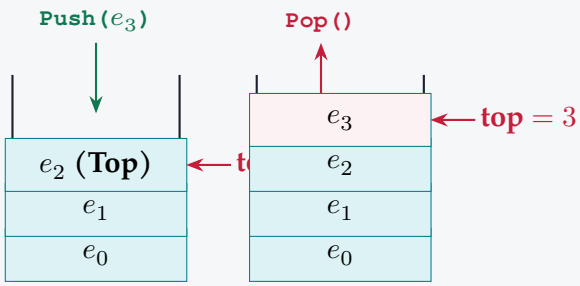
3. Algorithm Peek(S, top):

```
if (top == -1)
    report "Stack Empty" and exit error;
return S[top]; // Non-destructive inspection
```


© Biraj

Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

LIFO Mechanism and State Transitions in Stack Operations



2. Applications of Stacks

2.1 Arithmetic Expression Formats and Operator Precedence

Mathematical expressions are represented in three distinct syntactic notations:

1. **Infix Notation:** Binary operators are placed between operands: ⟨Operand⟩ **Op** ⟨Operand⟩ (e.g., $(A + B) * C$). Requires explicit parentheses and precedence parsing rules.
2. **Postfix Notation (Reverse Polish Notation — RPN):** Operators appear immediately following their operands: ⟨Operand⟩ ⟨Operand⟩ **Op** (e.g., $AB + C*$). Evaluated linearly without parentheses or backtrack searching.
3. **Prefix Notation (Polish Notation):** Operators precede operands: **Op** ⟨Operand⟩ ⟨Operand⟩ (e.g., $* + ABC$).

Operator	Operation Description	Associativity	Precedence Rank
`^` or `$`	**Exponentiation**	**Right-to-Left (R → L)**	Highest (Rank 3)
`*`, `/`, `%`	**Multiplication, Division, Modulo**	**Left-to-Right (L → R)**	Intermediate (Rank 2)
`+`, `-`	**Addition, Subtraction**	**Left-to-Right (L → R)**	Lowest (Rank 1)
`(`	**Left Parenthesis**	**Left-to-Right**	Rank 0 (Inside Stack)

© Biraj

2.2 Infix to Postfix Conversion Algorithm

Shunting-Yard Infix to Postfix Algorithm

1. Initialize an empty operator stack S and an empty postfix output string P.
2. Scan the infix expression string I from left to right, character by character:
   - **If Operand:** Append directly to postfix string P.
   - **If Left Parenthesis `'('`:** Push onto stack S.
   - **If Right Parenthesis `')'`:** Pop from S and append to P repeatedly until `'('` is popped. Discard both parentheses.
   - **If Operator** op**:** While stack is non-empty, and $\text{Prec}(\text{top}) > \text{Prec}(op)$ (or $\text{Prec}(\text{top}) == \text{Prec}(op)$ with Left-Associativity), pop S and append to P. Then push op onto S.
3. After end of input string, pop all remaining operators from S and append to P.

Step-by-Step Trace: Infix to Postfix Conversion

Convert: $I = A + B * (C \wedge D - E) \wedge (F + G * H) - I$

© Biraj

Token	Operator Stack	Postfix Output P	Action
A	$\emptyset$	A	**Append operand**
$+$	$+$	A	**Push operator**
B	$+$	AB	**Append operand**
$*$	$+\ *$	AB	**Push (higher precedence)**
$($	$+\ *\ ($	AB	**Push parenthesis**
C	$+\ *\ ($	ABC	**Append operand**
$\wedge$	$+\ *\ (\ \wedge$	ABC	**Push exponent**
D	$+\ *\ (\ \wedge$	$ABCD$	**Append operand**
$-$	$+\ *\ (\ -$	$ABCD\wedge$	**Pop $\wedge$, push $-$**
E	$+\ *\ (\ -$	$ABCD\wedge E$	**Append operand**
$)$	$+\ *$	$ABCD\wedge E-$	**Pop until $($, discard**
$\wedge$	$+\ *\ \wedge$	$ABCD\wedge E-$	**Push (higher precedence)**
$($	$+\ *\ \wedge\ ($	$ABCD\wedge E-$	**Push parenthesis**
F	$+\ *\ \wedge\ ($	$ABCD\wedge E-F$	**Append operand**
$+$	$+\ *\ \wedge\ (\ +$	$ABCD\wedge E-F$	**Push operator**
G	$+\ *\ \wedge\ (\ +$	$ABCD\wedge E-FG$	**Append operand**
$*$	$+\ *\ \wedge\ (\ +\ *$	$ABCD\wedge E-FG$	**Push (higher precedence)**
H	$+\ *\ \wedge\ (\ +\ *$	$ABCD\wedge E-FGH$	**Append operand**
$)$	$+\ *\ \wedge$	$ABCD\wedge E-FGH*+$	**Pop until $($, discard**
$-$	$-$	$ABCD\wedge E-FGH*+\wedge *+$	**Pop $\wedge, *, +$, push $-$**
I	$-$	$ABCD\wedge E-FGH*+\wedge *+I$	**Append operand**
End	$\emptyset$	**ABCD** $\wedge$ **E** $-$ **FGH** $*+\wedge *+$ **I** $-$	**Pop all remaining**

2.3 Evaluation of Postfix Expressions

Postfix Expression Evaluation Algorithm

Scan postfix string P from left to right:

1. If current symbol is an operand, push its numerical value onto operand stack S.
2. If symbol is an operator $\odot$:
   - $\text{op}_2 = \text{Pop}(S)$ (*Second operand popped first!*)
   - $\text{op}_1 = \text{Pop}(S)$ (*First operand popped second!*)
   - Compute result $= \text{op}_1 \odot \text{op}_2$.
   - $\text{Push}(S, \text{result})$.
3. When string ends, the final evaluated result resides at $\text{Pop}(S)$.

© Biraj

Time Complexity: $\mathcal{O}(\mathbf{n})$ Auxiliary Space: $\mathcal{O}(\mathbf{n})$

Evaluation Trace: Numerical Postfix String

Evaluate: $P = 6\ 2\ 3\ +\ *\ 12\ 4\ /\ -$

- Scan `6, 2, 3`: Push to stack $\implies S = [6, 2, 3]$.
- Encounter `+`: Pop $\text{op}_2 = 3$, $\text{op}_1 = 2$. Compute $2 + 3 = 5$. Push 5 $\implies S = [6, 5]$.
- Encounter `*`: Pop $\text{op}_2 = 5$, $\text{op}_1 = 6$. Compute $6 * 5 = 30$. Push 30 $\implies S = [30]$.
- Scan `12, 4`: Push to stack $\implies S = [30, 12, 4]$.
- Encounter `/`: Pop $\text{op}_2 = 4$, $\text{op}_1 = 12$. Compute $12/4 = 3$. Push 3 $\implies S = [30, 3]$.
- Encounter `-`: Pop $\text{op}_2 = 3$, $\text{op}_1 = 30$. Compute $30 - 3 = 27$. Push 27 $\implies S = [27]$.
- End of expression: Final Result = **27**.

2.4 Infix to Prefix Conversion and Prefix Evaluation

Infix to Prefix Conversion Algorithm (Reversal Method)

To convert an infix expression I to prefix notation Pre:
1. Reverse the infix string I.
2. In the reversed string, replace every `'('` with `')'` and every `')'` with `'('`.
3. Compute the postfix expression of this transformed string using the Shunting-Yard algorithm (with operator precedence rules: if precedence of scanned operator equals TOS, operators with right-associativity pop TOS, left-associative operators are pushed).
4. Reverse the resulting postfix string. The final string is the valid **Prefix expression**.

Time Complexity: $\mathcal{O}(\mathbf{n})$ Auxiliary Space: $\mathcal{O}(\mathbf{n})$

Example: Infix to Prefix Conversion

Convert: $I = (A + B) * C - (D - E) * (F + G)$
1. Reverse I: $(G + F) * (E - D) - C * (B + A)$
2. Swap brackets: $(G + F) * (E - D) - C * (B + A)$
3. Postfix of transformed string: $GF + ED - *CBA + *-$
4. Reverse result: $- * +$**ABC** $* -$**DE** $+$ **FG** (or $- * +ABC * -DE + FG$)

Prefix Expression Evaluation Algorithm

Scan the prefix expression Pre from **Right to Left**:
1. If current symbol is an operand, push its value onto operand stack S.
2. If symbol is an operator $\odot$:
   - $\text{op}_1 = \text{Pop}(S)$ (*First operand popped first!*)
   - $\text{op}_2 = \text{Pop}(S)$ (*Second operand popped second!*)
   - Compute result $= \text{op}_1 \odot \text{op}_2$.
   - Push(S, result).

© Biraj

3. When scan completes, Pop(S) contains the final evaluated scalar value.

2.5 Balanced Delimiter Checking and Call Stack Simulation

Algorithm: Delimiter Matching (Parentheses Balancing)

Algorithm CheckBalanced$(expr)$**:**
1. Initialize an empty character stack S.
2. For each character ch in string $expr$:
   - If $ch \in \{$`'('`, `'['`, `'{'`$\}$, execute Push(S, ch).
   - If $ch \in \{$`')'`, `']'`, `'}'`$\}$:
     - If IsEmpty(S), return **False** (Underflow: unmatched closing bracket).
     - $topChar =$ Pop(S).
     - If $(topChar, ch)$ is not a matching pair `()`, `[]`, or `{}`, return **False**.
3. Return IsEmpty(S) (True if all opened brackets were closed; False otherwise).

Time Complexity: $\mathcal{O}(\mathbf{n})$ Space Complexity: $\mathcal{O}(\mathbf{n})$

Function Call Stack and Recursion Architecture

When a recursive function (such as $n!$ or Fibonacci) executes, the operating system instantiates an **Activation Record (Stack Frame)** on the thread's call stack for each recursive call:
- Contents of a Frame: Return Address, Function Parameters, Local Variables, Saved Base Pointer (`EBP/RBP`).
- Growth: As calls nest deeper, frames stack vertically. Winding phase consumes $\mathcal{O}(n)$ stack space.
- Unwinding: Upon reaching base case, frames pop in LIFO sequence, returning values to caller.

3. Abstract Data Type Queue and Linear Queue

Definition: Queue Abstract Data Type (ADT)

A **Queue** is a linear, homogeneous data structure governed by the **First-In, First-Out (FIFO)** operating principle. Insertions are restricted to the **Rear** terminal, while deletions occur exclusively at the **Front** terminal.

Pointers in an array of size MAX:
- front: Points to the position of the first valid data element (or one position prior in alternate textbook conventions).
- rear: Points to the index of the latest inserted data item.

3.1 Array Implementation of Linear Queue

Let array storage be $Q[0 \ldots \text{MAX} - 1]$ with standard 0-based indices:

- **Initial Empty State:** front $= -1,$ rear $= -1.$

© Biraj

- **Queue Underflow Condition:** front $== -1$ (or front $>$ rear).
- **Queue Overflow Condition:** rear $==$ MAX $- 1$.

Linear Queue Algorithms and the False Overflow Anomaly

1. Algorithm Enqueue(Q, x):

```
if (rear == MAX - 1)
    report "Queue Overflow" and exit error;
if (front == -1) front = 0; // First element insertion
rear = rear + 1;
Q[rear] = x;
```

2. Algorithm Dequeue(Q):

```
if (front == -1 || front > rear)
    report "Queue Underflow" and exit error;
item = Q[front];
front = front + 1;
if (front > rear) // Reset queue when emptied
    front = -1, rear = -1;
return item;
```

The False Overflow Problem (Memory Drift): In a simple linear queue, after several enqueue and dequeue cycles, rear reaches MAX $- 1$ while front has advanced forward (e.g., front $= 4$, rear $=$ MAX $- 1$). Even though indices $0 \ldots 3$ are physically empty, the condition rear $==$ MAX $- 1$ reports overflow and rejects new insertions!

4. Advanced Queue Architectures

4.1 Circular Queue

A **Circular Queue** overcomes false overflow by logically connecting index MAX $- 1$ back to index 0 via **modulo arithmetic**:

$$\text{next}(\text{index}) = (\text{index} + 1) \bmod \text{MAX} \tag{3}$$

Circular Queue Boundary States and Algorithms

Let $Q[0 \ldots \text{MAX} - 1]$:

- **Initial Empty State:** front $= -1,$ rear $= -1$.
- **Circular Queue Full Condition:**

$$(\text{rear} + 1) \bmod \text{MAX} == \text{front} \tag{4}$$

- **Circular Queue Empty Condition:**

$$\text{front} == -1 \tag{5}$$

© Biraj

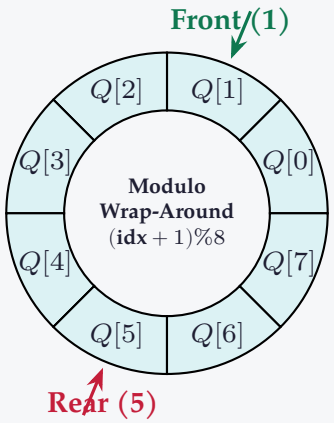
Algorithm CircularEnqueue(Q, x):

```
if ((rear + 1) % MAX == front)
    report "Circular Queue Overflow" and exit;
if (front == -1) // First element
    front = 0, rear = 0;
else
    rear = (rear + 1) % MAX;
Q[rear] = x;
```

Algorithm CircularDequeue(Q):

```
if (front == -1)
    report "Circular Queue Underflow" and exit;
item = Q[front];
if (front == rear) // Only one element was present
    front = -1, rear = -1;
else
    front = (front + 1) % MAX;
return item;
```

Ring Representation of Circular Queue ($MAX = 8$)



4.2 Priority Queue

A **Priority Queue** is an ADT where each element has an associated priority key. Elements are processed according to priority rather than arrival time:

- **Ascending Priority Queue:** Item with the lowest numerical priority value is extracted first.
- **Descending Priority Queue:** Item with the highest numerical priority value is extracted first.

© Biraj

Implementation	Enqueue Time	Dequeue Time	Trade-off Remarks
Unordered Array	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Linear search required to find extreme priority.
Ordered Array	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Sorted insertion requires element shifts.
Unordered Linked List	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Deletion requires traversing entire list.
Ordered Linked List	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Dequeue is $\mathcal{O}(1)$ removal at head.
Binary Heap	$\mathcal{O}(\log \mathbf{n})$	$\mathcal{O}(\log \mathbf{n})$	**Optimal engineering balance** (standard in C++/Java).

4.3 Double-Ended Queue (DEQUE)

A **DEQUE** (Double-Ended Queue) permits insertion and deletion at both the front and rear ends:

1. **Input-Restricted DEQUE:** Insertion is allowed at only one end (e.g., rear), while deletion is allowed at both ends (front and rear).
2. **Output-Restricted DEQUE:** Deletion is allowed at only one end (e.g., front), while insertion is allowed at both ends (front and rear).

© Biraj

Quick Revision — Module 2

Stacks and Queues: ADTs, Operations and Applications | ES-CS301

Key Formulas and Boundary Conditions at a Glance

Stack Overflow Condition	$\text{top} == \text{MAX} - 1$
Stack Underflow Condition	$\text{top} == -1$
Linear Queue Empty	$\text{front} == -1 \quad \text{or} \quad \text{front} > \text{rear}$
Linear Queue Overflow	$\text{rear} == \text{MAX} - 1$ (Susceptible to False Overflow)
Circular Queue Next Pointer	$\text{next} = (\text{index} + 1) \bmod \text{MAX}$
Circular Queue Full	$(\text{rear} + 1) \bmod \text{MAX} == \text{front}$
Circular Queue Empty	$\text{front} == -1$
Circular Queue Element Count	$\text{Count} = (\text{rear} - \text{front} + \text{MAX}) \bmod \text{MAX} + 1$ (if non-empty)
Stack Operation Complexities	Push: $\mathcal{O}(1)$, Pop: $\mathcal{O}(1)$, Peek: $\mathcal{O}(1)$
Binary Heap Priority Queue	Insert: $\mathcal{O}(\log n)$, Extract-Min/Max: $\mathcal{O}(\log n)$

One-Line Definitions

- **Stack:** A linear list where insertions and deletions occur strictly at one end (LIFO discipline).
- **Queue:** A linear list where insertions occur at the rear and deletions at the front (FIFO discipline).
- **Top of Stack (TOS):** An integer pointer referencing the most recently pushed element.
- **False Overflow:** Rejection of enqueue requests in a linear queue when rear is at $\text{MAX} - 1$ despite empty slots at the front.
- **Circular Queue:** A queue where the end of the array wraps around to the beginning using modulo arithmetic.
- **Priority Queue:** A queue where elements are retrieved based on an assigned priority key rather than FIFO order.
- **DEQUE:** A generalized queue permitting insertion and deletion operations at both extremities.
- **Infix Notation:** Arithmetic notation placing operators between operands (requires parentheses).
- **Postfix Notation (RPN):** Arithmetic notation placing operators after operands (parenthesis-free).

© Biraj

- **Prefix Notation:** Arithmetic notation placing operators before operands (Polish notation).
- **Activation Record:** A memory block pushed onto the call stack containing parameters, local variables, and return addresses.
- **Balanced Delimiters:** A string of grouping symbols where each opening token matches its corresponding closing token in proper nested order.

Frequently Asked / Viva Questions

Q1. What is the primary difference between LIFO and FIFO operating principles?

LIFO (Last-In, First-Out), implemented by Stacks, ensures that the most recently inserted item is the first to be retrieved. FIFO (First-In, First-Out), implemented by Queues, guarantees that elements are serviced strictly in chronological order of arrival.

Q2. Why are Postfix expressions preferred over Infix expressions by compiler evaluators?

Infix expressions require operator precedence tables, associativity rules, and parenthetical parsing, necessitating lookaheads and recursive evaluation. Postfix expressions are completely parenthesis-free and can be evaluated in a single linear pass ($\mathcal{O}(n)$ time) using an operand stack.

Q3. How does a Circular Queue solve the False Overflow issue of a Linear Queue?

In a linear queue, deleted slots before `front` cannot be reused once `rear` reaches $\text{MAX} - 1$. A circular queue treats the array as a closed ring using modulo arithmetic ($(\text{rear} + 1) \bmod \text{MAX}$), wrapping new insertions into vacant slots at the beginning of the buffer.

Q4. During postfix evaluation, why must the first popped element be assigned as the second operand?

Because stack operations are LIFO. If $A-B$ is converted to $AB-$, B is pushed after A and is therefore on top. Popping retrieves B first (op_2) and A second (op_1). For non-commutative operators (subtraction, division), swapping operands yields incorrect results ($\text{op}_1 - \text{op}_2 \neq \text{op}_2 - \text{op}_1$).

Q5. State the condition for a Circular Queue of size MAX to be completely full.

The standard condition is $(\text{rear} + 1) \bmod \text{MAX} == \text{front}$. This ensures that advancing the rear pointer by one slot would collide with the current front position.

Q6. What is the difference between an Input-Restricted and an Output-Restricted DEQUE?

In an Input-Restricted DEQUE, insertions are permitted at only one end (e.g., rear), while deletions can occur at both ends (front and rear). In an Output-Restricted DEQUE, deletions are restricted to one end (e.g., front), while insertions can take place at both ends.

© Biraj

Frequently Asked / Viva Questions (contd.)

Q7. How can a stack be used to check for balanced parentheses in an expression?

Traverse the expression character by character. When an opening delimiter (`(`, `[`, `{`) is found, push it onto the stack. When a closing delimiter (`)`, `]`, `}`) is found, pop the stack and check if it matches the closing symbol. If underflow occurs or types mismatch, it is unbalanced. The expression is balanced if the stack is empty at the end.

Q8. Why is a Binary Heap the preferred data structure for implementing a Priority Queue?

In an array or linked list, either insertion or extraction is $\mathcal{O}(n)$. A Binary Heap achieves an optimal compromise of $\mathcal{O}(\log n)$ for both insertion and extraction of the extreme-priority item, making it ideal for dynamic workloads.

Q9. What happens to the call stack during infinite recursion, and what error is generated?

Each recursive call allocates an activation frame on the runtime call stack. Without a valid terminating base condition, stack frames accumulate until memory allocated for the call stack is exhausted, triggering a fatal **Stack Overflow** error.

Q10. Can two stacks be implemented efficiently within a single one-dimensional array?

Yes. Stack 1 starts at index 0 and grows rightward ($\text{top}_1 ++$), while Stack 2 starts at index $\text{MAX} - 1$ and grows leftward ($\text{top}_2 --$). Overflow occurs only when $\text{top}_1 + 1 == \text{top}_2$, maximizing total array space utilization.

© Biraj

Summary Matrix: Comprehensive Comparison of Stack and Queue Structures

Structure	Discipline / Pointers	Critical Conditions	Primary Practical Applications
Stack **Single pointer top** **Empty:** $\text{top} == -1$	**LIFO / FILO;** **Full:** $\text{top} == \text{MAX} - 1$	**Expression parsing, recursion, backtracking, undo operations.**	
Linear Queue **Two pointers front, rear** **Empty:** $\text{front} == -1$	**FIFO / LILO;** **Full:** $\text{rear} == \text{MAX} - 1$	**Print spooling, simple task buffering (suffers memory drift).**	
Circular Queue **Modulo wrap-around** **Empty:** $\text{front} == -1$	**Ring FIFO;** **Full:** $(\text{rear} + 1)\%\text{MAX} == \text{front}$	**CPU scheduling (round robin), IO buffering, audio streaming.**	
Priority Queue **Key-based selection**	**Priority-ordered;** **Governed by underlying heap/array capacity**	**Dijkstra's algorithm, Huffman coding, OS interrupt processing.**	
DEQUE **Front & Rear access**	**Double-ended;** **Full: depends on circular or linear array implementation**	**Browser tab history, job-stealing algorithms, palindrome verification.**	

© Biraj